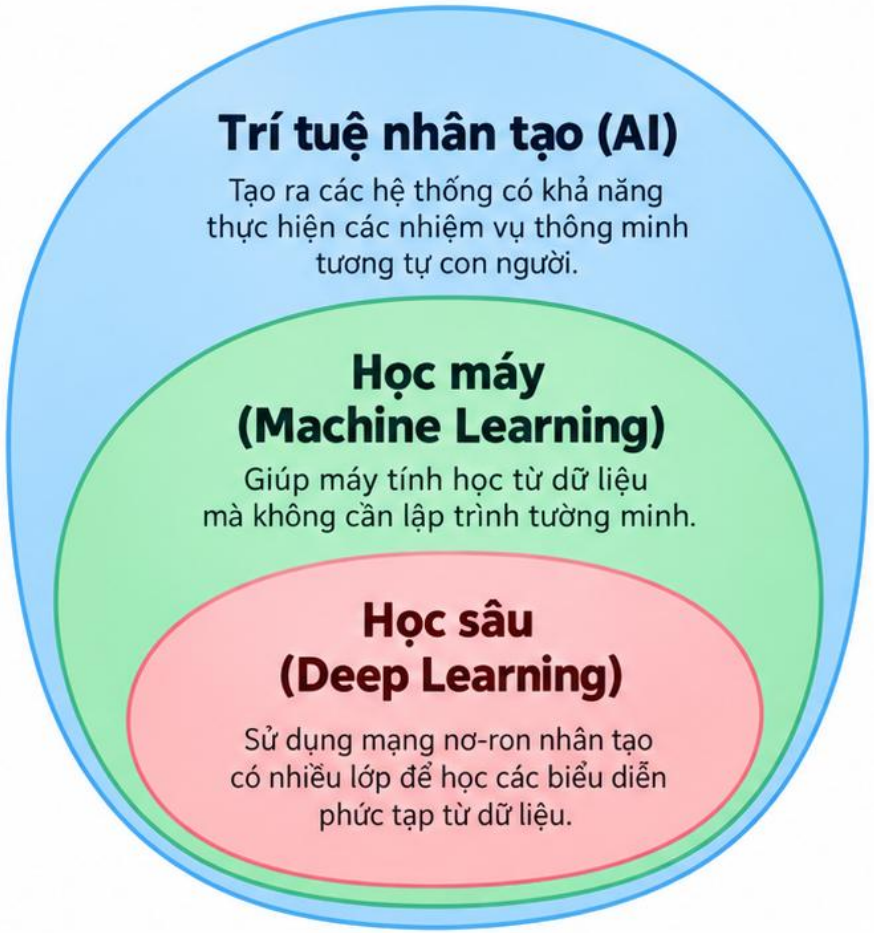
HỌC SÂU
DEEP LEARNING

Vị trí của Học sâu trong AI



Học sâu là một nhánh của Học máy, và Học máy là một nhánh của AI.

Một số bài toán tiêu biểu của học sâu



Phân loại ảnh

Nhận biết và phân loại đối tượng trong ảnh.



Nhận dạng khuôn mặt

Xác định và nhận diện khuôn mặt của người.



Nhận dạng giọng nói

Chuyển đổi giọng nói thành văn bản và nhận diện người nói.



Xử lý ngôn ngữ tự nhiên

Hiểu và sinh ngôn ngữ của con người.



Dịch máy

Dịch văn bản giữa các ngôn ngữ khác nhau.



Xe tự hành / thị giác máy tính

Nhận biết môi trường, phát hiện vật thể và hỗ trợ xe tự hành.

Phù hợp với dữ liệu phức tạp như ảnh, âm thanh, văn bản, video.

1. Mục tiêu học phần

- Giải thích được nguyên lý mạng nơ-ron, forward propagation, loss, gradient và backpropagation.
- Phân biệt vai trò của MLP, CNN, RNN/LSTM, Transformer và Deep Reinforcement Learning.
- Xây dựng, huấn luyện, đánh giá và tinh chỉnh mô hình bằng PyTorch ở mức cơ bản
- Phân tích kết quả thực nghiệm dựa trên metric, learning curve và lỗi thường gặp.
- Làm việc nhóm, báo cáo bài tập lớn và sử dụng công nghệ học sâu có trách nhiệm.

2. Cách tổ chức 10 buổi học (50 tiết)

- Buổi 1–2: nền tảng neural network, MLP, loss, backpropagation và training loop.
- Buổi 3–4: học sâu cho ảnh với CNN, đánh giá, augmentation và transfer learning.
- Buổi 5–7: dữ liệu chuỗi/văn bản, RNN/LSTM, attention, Transformer và fine-tuning.
- Buổi 8–9: Deep RL từ MDP, Q-learning, DQN đến policy gradient và actor-critic.
- Buổi 10: regularization, tái lập thực nghiệm, tổng kết và báo cáo bài tập lớn.

3. Đánh giá học phần và sản phẩm học tập

- A1.1 – Bài thực hành/trả lời: 10%,
- A1.2 – Bài thực hành/thuyết trình: 30%,
- A2 – Bài tập lớn: 60%, gồm mã nguồn, báo cáo, demo và thuyết trình nhóm.
- Điểm không chỉ dựa vào accuracy; cần có baseline, cấu hình, metric và phân tích lỗi.

Team code: N1: 36k0xhi

4.Quy tắc học thực hành

- Chạy baseline trước, lưu kết quả gốc rồi mới thay đổi kiến trúc/hyperparameter.
- Mỗi thay đổi cần có giả thuyết: vì sao thay đổi đó có thể cải thiện hoặc làm xấu mô hình.
- Đọc learning curve để phân biệt underfitting, overfitting và lỗi cấu hình.
- Khi debug, kiểm tra theo thứ tự: dữ liệu → shape tensor → loss → optimizer → metric.
- Bài thực hành/luyện tập sử dụng dataset có sẵn để tập trung vào phương pháp thay vì thu thập dữ liệu.

5. Môi trường thực hành

• Server/Colab/Kaggle..: phù hợp cho sinh viên chưa có GPU; bật Runtime → GPU khi dùng CNN/Transformer.
• Máy cá nhân: dùng Python 3.10+, virtual environment và file requirements.txt.
  - **torch**: thư viện cốt lõi của PyTorch để xây dựng, huấn luyện và tính toán mạng nơ-ron.
  - **torchvision**: hỗ trợ dữ liệu ảnh, mô hình thị giác, phép biến đổi ảnh và các dataset phổ biến.
  - **numpy**: xử lý mảng số, vector, ma trận và các phép tính số học hiệu quả.
  - **pandas**: đọc, xử lý, làm sạch và phân tích dữ liệu dạng bảng.
  - **matplotlib**: vẽ biểu đồ, trực quan hóa dữ liệu và kết quả huấn luyện.
  - **scikit-learn**: cung cấp các thuật toán Machine Learning, tiền xử lý dữ liệu và các metric đánh giá.
  - **gymnasium[classic-control]**: cung cấp các môi trường học tăng cường như CartPole, MountainCar.
  - **datasets**: tải và xử lý các bộ dữ liệu chuẩn, đặc biệt tiện cho NLP.
  - **transformers**: sử dụng và fine-tune các mô hình Transformer pretrained như BERT, GPT, T5.
  - **accelerate**: hỗ trợ huấn luyện mô hình dễ dàng trên CPU/GPU và nhiều thiết bị.

5.Môi trường thực hành- Dữ liệu

• Dataset dùng xuyên suốt: FashionMNIST, CIFAR-10, AG News, FrozenLake, CartPole.

Dataset	Bài toán	Thời gian
FashionMNIST	Phân loại ảnh quần áo 28×28 vào 10 lớp; dùng để minh họa MLP, forward/backpropagation và quy trình huấn luyện cơ bản.	Buổi 1–2
CIFAR-10	Phân loại ảnh màu 32×32 vào 10 lớp; dùng để học CNN, đánh giá mô hình, data augmentation và transfer learning.	Buổi 3–4
AG News	Phân loại bài báo vào 4 chủ đề; dùng để học embedding, RNN/LSTM và fine-tuning Transformer cho văn bản.	Buổi 5–7
FrozenLake	Agent tìm đường đến đích trên lưới, tránh ô nguy hiểm; dùng để minh họa MDP, state, action, reward và Q-learning.	Buổi 8
CartPole	Agent điều khiển xe sang trái/phải để giữ thanh cân bằng; dùng để minh họa DQN, policy gradient và Deep RL.	Buổi 8–9

BUỔI 1

Tổng quan học sâu, perceptron và MLP

- Hiểu vị trí của Deep Learning trong AI/ML và các bài toán thực tế.
- Nắm cấu trúc neuron nhân tạo, perceptron, hàm kích hoạt và MLP.
- Biết đọc shape đầu vào/đầu ra và tư duy forward propagation.
- Thực hành phân loại dữ liệu tabular nhỏ bằng MLP (Multi-Layer Perceptron - mạng Perceptron nhiều lớp).

1. AI, Machine Learning và Deep Learning

• AI là mục tiêu rộng: tạo hệ thống có hành vi thông minh trong một nhiệm vụ.
• Machine Learning học quy luật từ dữ liệu thay vì viết luật thủ công cho từng trường hợp.
• Deep Learning dùng nhiều tầng biến đổi để học biểu diễn từ dữ liệu thô.
• Trong thực tế, mô hình tốt cần dữ liệu phù hợp, metric đúng và quy trình đánh giá rõ ràng.
• Câu hỏi : Bài toán nào cần deep learning, bài toán nào chỉ cần mô hình đơn giản?

Dùng mô hình đơn giản	Dùng Deep Learning
Linear/Logistic Regression, Decision Tree, Random Forest, XGBoost khi dữ liệu chủ yếu là **bảng số liệu có cấu trúc**, số lượng dữ liệu không quá lớn, và quan hệ giữa đầu vào–đầu ra không quá phức tạp. Ví dụ: dự đoán điểm sinh viên, dự đoán giá nhà, phân loại khách hàng rời bỏ dịch vụ.	Dữ liệu **phức tạp và nhiều chiều** như ảnh, âm thanh, video, văn bản dài, hoặc khi cần mô hình tự học đặc trưng thay vì phải thiết kế đặc trưng thủ công. Ví dụ: nhận diện khuôn mặt, phân loại ảnh y tế, nhận dạng giọng nói, dịch máy, chatbot, tóm tắt văn bản.

2. Tư duy mô hình hóa trong học sâu

- Mọi mô hình học sâu đều có ba thành phần cốt lõi: **dữ liệu, hàm dự đoán và hàm mất mát.**
- Dữ liệu được biểu diễn thành tensor; sai shape là lỗi phổ biến nhất khi mới học.
- Mô hình sinh dự đoán; loss đo mức sai; optimizer cập nhật tham số để giảm loss.
- Không học thuộc công thức rời rạc: luôn hỏi biến đó có shape gì và ý nghĩa thực tế gì.
- Liên hệ: phân loại email rác, nhận dạng ảnh, phân loại cảm xúc.
    - **Phân loại email rác:** Email → mô hình dự đoán Spam/Không spam → loss đo sai lệch với nhãn thật.
    - **Nhận dạng ảnh:** Ảnh → mô hình dự đoán lớp đối tượng → loss đo sai lệch giữa lớp dự đoán và lớp đúng.
    - **Phân loại cảm xúc:** Văn bản → mô hình dự đoán Tích cực/Trung tính/Tiêu cực → loss đo sai lệch với nhãn cảm xúc thật.

3. AI, Machine Learning và Deep Learning – Ứng dụng và triển khai

Ví dụ thực tế: Kiểm tra chất lượng sản phẩm

- Đầu vào (x): Ảnh sản phẩm từ camera.
- Đầu ra (y): Nhãn “Đạt” hoặc “Lỗi”.
- Deep Learning: Tự học đặc trưng như cạnh, vết xước, vùng biến màu để phân loại sản phẩm.

Quy trình triển khai

- Xác định dữ liệu đầu vào, nhãn cần dự đoán, metric và chi phí dự đoán sai.
- Kiểm tra dữ liệu trước khi huấn luyện: shape → forward → loss → gradient.
- Huấn luyện và đánh giá trên tập dữ liệu độc lập.
- Bắt đầu bằng mô hình đơn giản, chỉ tăng độ phức tạp khi cần thiết.

Lỗi thường gặp

- Nhầm AI đồng nghĩa với Deep Learning.
- Cho rằng mạng càng nhiều tầng càng tốt.
- Chỉ đánh giá trên tập huấn luyện.
- Bỏ qua chi phí dữ liệu và rủi ro sai lệch.

4. Nguyên tắc thực nghiệm và phân tích kết quả

Phân tích sau huấn luyện

- Ghi rõ cấu hình thay đổi: số tầng, neuron, learning rate, regularization…
- So sánh metric trước/sau và chi phí thời gian, bộ nhớ.
- Quan sát learning curve, confusion matrix và các mẫu dự đoán sai.
- Không kết luận mô hình tốt hơn chỉ từ một lần chạy có chênh lệch nhỏ.

Nguyên tắc thực nghiệm

- Chỉ thay đổi một yếu tố chính tại một thời điểm (ablation).
- Giữ lại cấu hình, seed, kết quả và nhật ký huấn luyện.
- Sử dụng cùng dữ liệu đánh giá để so sánh công bằng.

Câu hỏi thảo luận

Nếu giảm số neuron từ 128 xuống 32, kết quả có thể thay đổi thế nào?
Vì sao mô hình có accuracy 99% trên tập train vẫn có thể hoạt động kém?
Ảnh sản phẩm chụp trong phòng thí nghiệm có đại diện cho ảnh tại nhà máy không?

Công thức dự đoán:

$$\boxed{\hat{y} = f_\theta(x)}$$

Công thức tối ưu hóa mô hình:

$$\boxed{\theta^* = \arg\min_\theta \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(f_\theta(x_i), y_i\right)}$$

Ký hiệu	Ý nghĩa
x	Dữ liệu đầu vào, ví dụ ảnh, văn bản hoặc đặc trưng sinh viên.
f_θ	Mô hình học sâu với các tham số θ.
θ	Các tham số cần học của mô hình, chủ yếu gồm trọng số W và độ lệch b.
$\hat{y}$	Giá trị dự đoán của mô hình.
y_i	Nhãn hoặc giá trị thực tế của mẫu thứ i.
n	Tổng số mẫu dữ liệu huấn luyện.
$\mathcal{L}$	Hàm mất mát (Loss Function), đo mức sai lệch giữa dự đoán và thực tế.
$\sum_{i=1}^{n}$	Cộng giá trị mất mát của tất cả các mẫu.
$\arg\min_\theta$	Tìm bộ tham số θ làm cho giá trị mất mát trung bình nhỏ nhất.
θ^*	Bộ tham số tối ưu tìm được theo mục tiêu huấn luyện.

1. Dự đoán

$\hat{y} = f_\theta(x)$

→

2. Tối ưu

Giảm hàm mất mát để cải thiện dự đoán

3. Ví dụ thực tế: Phân loại email rác

Giả sử huấn luyện mô hình trên 1.000 email:

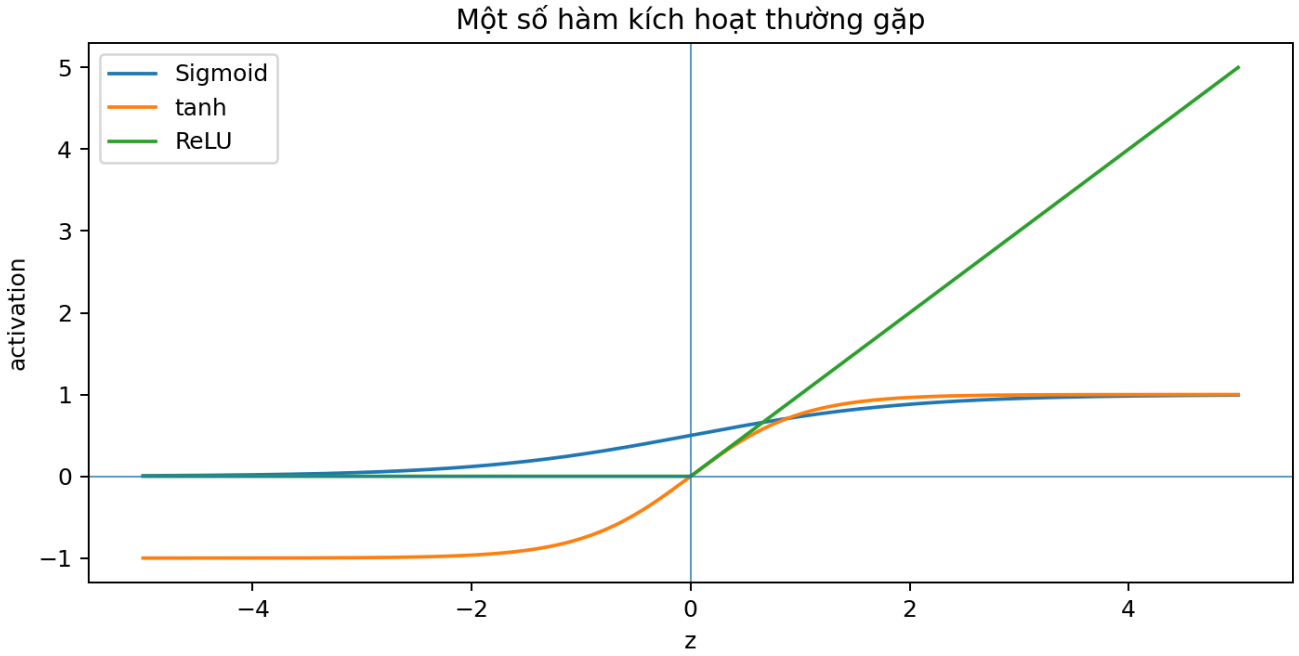
- **Đầu vào x:** nội dung email.
- **Đầu ra $\hat{y}$:** dự đoán email rác hay không rác.
- **Nhãn thực y:** kết quả phân loại đúng do con người cung cấp.
- **Hàm mất mát $\mathcal{L}$:** đo mức độ sai của dự đoán.
- **Quá trình tối ưu:** điều chỉnh trọng số mô hình để giảm mức mất mát trung bình trên 1.000 email.

5. Perceptron và neuron nhân tạo

- Một neuron nhận vector đầu vào x, nhân với trọng số W, cộng bias b và đưa qua hàm kích hoạt.
- Phép tuyến tính z = Wx + b tạo ra điểm số thô trước khi phi tuyến hóa.
- Bias cho phép dịch ranh giới quyết định; trọng số quyết định mức ảnh hưởng của từng đặc trưng.
- Perceptron tuyến tính chỉ tách tốt các lớp có ranh giới tuyến tính.
- MLP mở rộng perceptron bằng nhiều tầng và hàm kích hoạt phi tuyến.

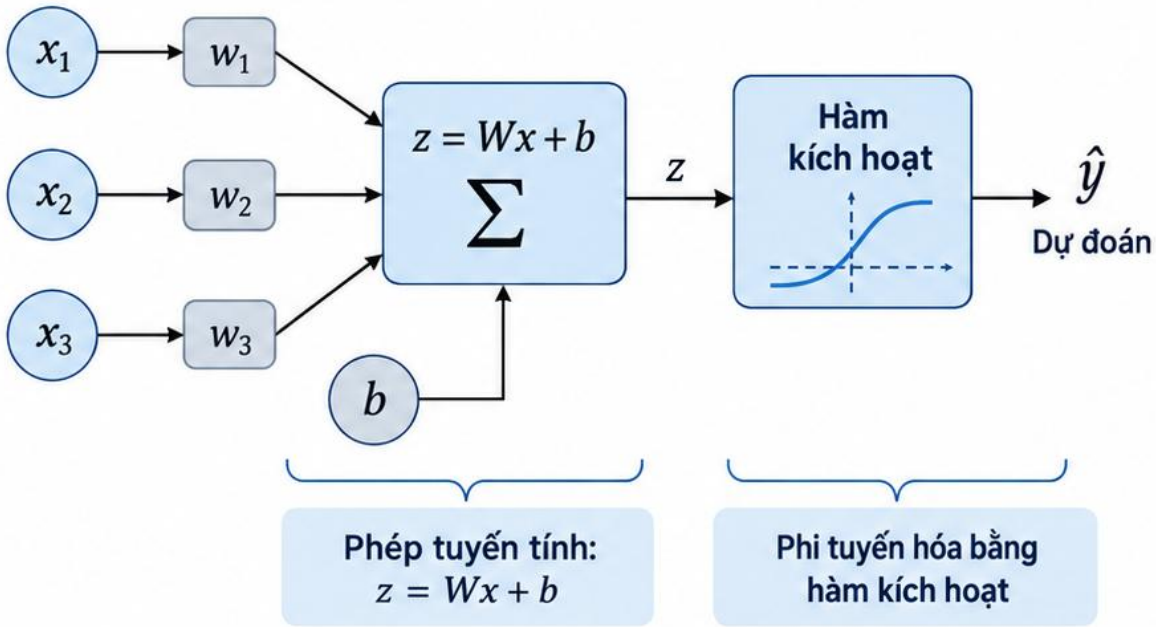
6. Hàm kích hoạt

- Nếu không có hàm kích hoạt, nhiều tầng tuyến tính vẫn tương đương một tầng tuyến tính.
- Sigmoid hữu ích cho xác suất nhị phân nhưng dễ bão hòa gradient ở vùng xa 0.
- tanh có miền giá trị [-1, 1] nhưng vẫn gặp bão hòa gradient.
- ReLU đơn giản, nhanh và thường hiệu quả: f(z)=max(0,z).
- Khi ReLU cho toàn 0 kéo dài, cần kiểm tra learning rate và khởi tạo trọng số.



Hình minh họa từ giáo trình

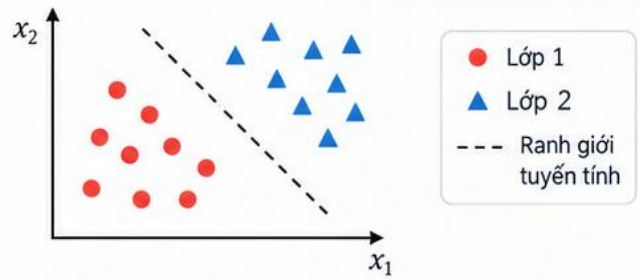
Perceptron và neuron nhân tạo



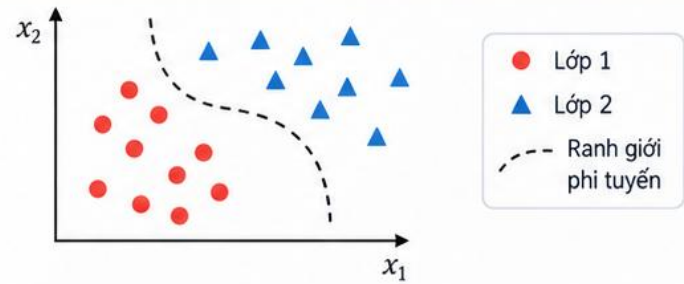
Một số điểm quan trọng

1. Trọng số quyết định mức ảnh hưởng của từng đặc trưng.
2. **Bias** giúp dịch chuyển ranh giới quyết định.
3. **Perceptron tuyến tính** chỉ tách tốt dữ liệu có ranh giới tuyến tính.
4. **MLP** mở rộng perceptron bằng nhiều tầng và hàm kích hoạt phi tuyến.

Perceptron



MLP



Một neuron nhận đầu vào x, nhân với trọng số W, cộng bias b, rồi đưa qua hàm kích hoạt để tạo đầu ra.

Hàm kích hoạt	Tại sao sử dụng?	Mạng thường dùng	Bài toán ứng dụng cụ thể
Sigmoid	Đưa đầu ra về [0,1], phù hợp biểu diễn xác suất	Lớp output của MLP – Multi-Layer Perceptron, CNN – Convolutional Neural Network	Spam/không spam; bệnh/không bệnh; gian lận/không gian lận
tanh	Đưa giá trị về [-1,1], phù hợp biểu diễn trạng thái có cả âm và dương	RNN – Recurrent Neural Network, LSTM – Long Short-Term Memory	Phân loại cảm xúc, dự báo chuỗi thời gian, xử lý câu/văn bản tuần tự
ReLU \(f(z)=\max(0,z)\)	Đơn giản, nhanh, giúp mạng sâu học hiệu quả	Lớp ẩn của MLP – Multi-Layer Perceptron, CNN – Convolutional Neural Network	Phân loại ảnh, nhận dạng đối tượng, dự đoán từ dữ liệu nhiều đặc trưng

MLP và lan truyền tiến — mô hình hóa và công thức

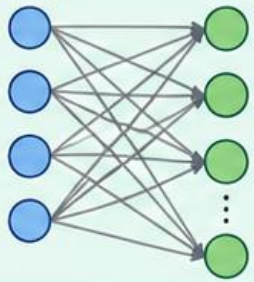
Đầu vào x
(vector đầu vào)

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

$x \in \mathbb{R}^d$

→

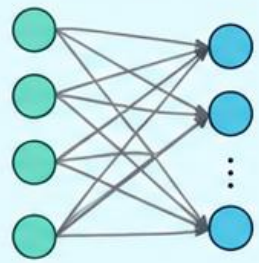
Lớp ẩn thứ nhất h_1
(128, ... neuron)



$$h_1 = \phi(W_1 x + b_1)$$

→

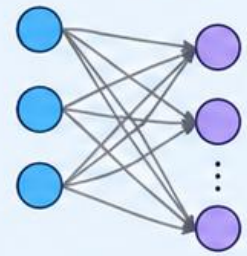
Lớp ẩn thứ hai h_2
(64, ... neuron)



$$h_2 = \phi(W_2 h_1 + b_2)$$

→

Đầu ra (logits) z
(số lớp: C)



$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_C \end{bmatrix}$$

$z \in \mathbb{R}^C$

$$\text{logits} = z = W_3 h_2 + b_3$$

Tất cả tham số của mô hình

$$\theta = \{W_1, b_1, W_2, b_2, W_3, b_3\}$$

θ gồm toàn bộ trọng số và bias của các lớp.

Giải thích ngắn gọn

- x: vector đầu vào.
- W, b: trọng số và bias; chúng quyết định cách mô hình biến đổi dữ liệu.
- ϕ: hàm kích hoạt (ví dụ ReLU, tanh) để tạo tính phi tuyến.
- h_1, h_2: biểu diễn trung gian mà mô hình học được.
- z **(logits):** điểm số thô cho từng lớp; thường đưa qua softmax để ra xác suất.
- **Shape** phải tương thích; số chiều của z bằng số lớp trong bài toán phân loại nhiều lớp.

Ví dụ ngắn

Phân loại ảnh FashionMNST thành 10 lớp



Ảnh 28 × 28

Trải phẳng →

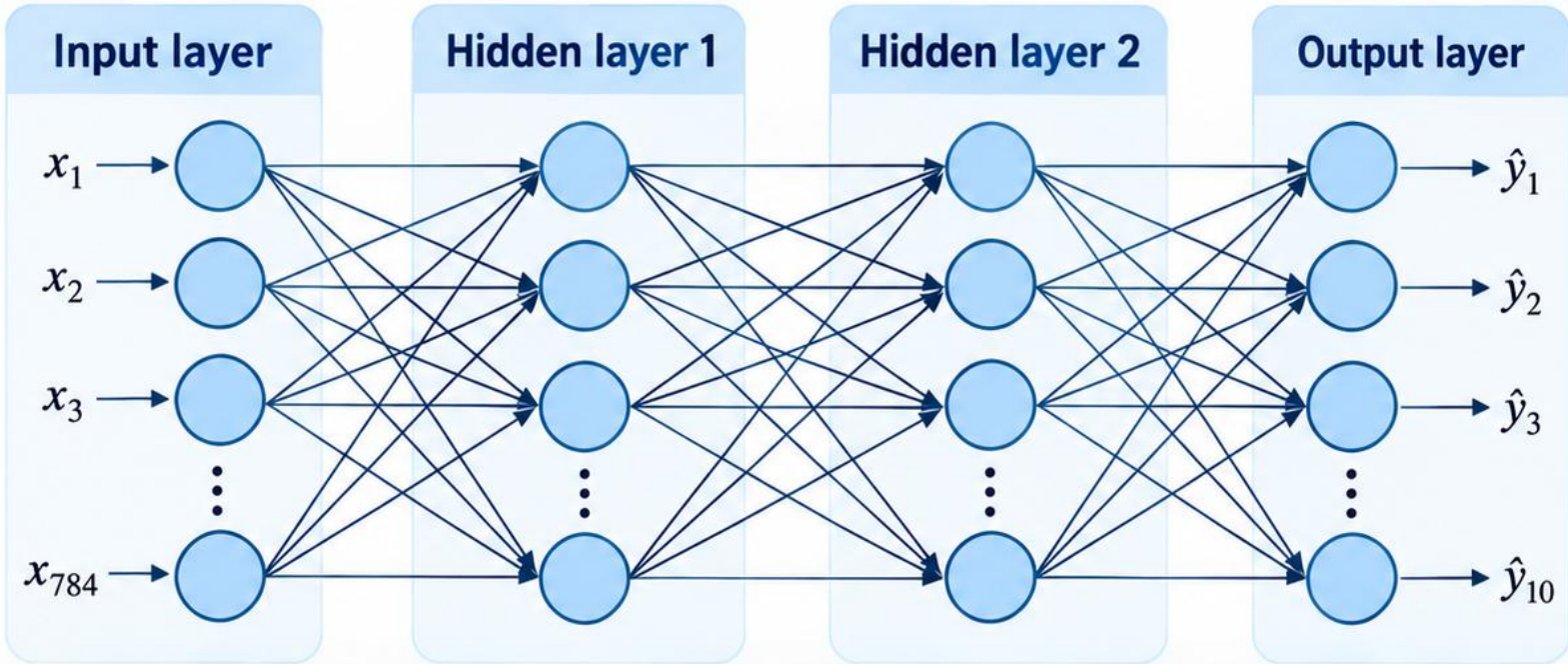
$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{784} \end{bmatrix}$$

$x \in \mathbb{R}^{784}$

- Ảnh 28 × 28 được trải phẳng thành $x \in \mathbb{R}^{784}$.
- Chọn 2 lớp ẩn: 128 và 64 neuron.
- Khi đó:
  - $W_1 \in \mathbb{R}^{128\times784}$, $b_1 \in \mathbb{R}^{128}$, $h_1 \in \mathbb{R}^{128}$
  - $W_2 \in \mathbb{R}^{64\times128}$, $b_2 \in \mathbb{R}^{64}$, $h_2 \in \mathbb{R}^{64}$
  - $W_3 \in \mathbb{R}^{10\times64}$, $b_3 \in \mathbb{R}^{10}$, $z \in \mathbb{R}^{10}$
- Phần tử lớn nhất trong z cho biết lớp dự đoán.

Ý chính: lan truyền tiến là quá trình biến đổi dữ liệu từ $x \to h_1 \to h_2 \to z$ để tạo dự đoán.

MLP và lan truyền tiến



Forward propagation: tính dự đoán từ input đến output, chưa cập nhật tham số

Ví dụ kiến trúc MLP

784 → 256 → 128 → 10

Ảnh 28×28 được flatten thành 784 đặc trưng

Công thức của một lớp

$$h = \sigma(Wx + b)$$

Output của lớp trước là input của lớp sau.

Ghi nhớ

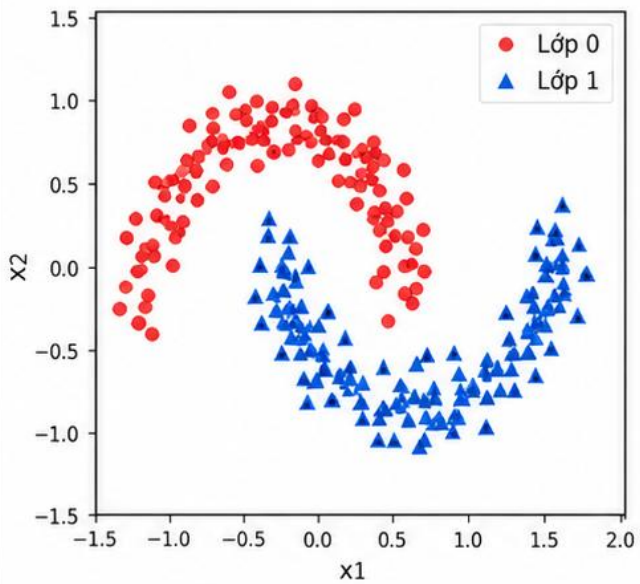
Khi thiết kế MLP, cần xác định rõ: **input_dim**, **hidden_dim**, **output_dim**.

Ví dụ: Phân loại điểm dữ liệu 2D

- Dữ liệu make_moons có ranh giới cong nên logistic regression tuyến tính thường khó tách tốt.
- MLP với một tầng ẩn có thể học ranh giới phi tuyến thông qua tổ hợp nhiều neuron.
- Không nên chỉ nhìn accuracy; cần quan sát loss và ranh giới quyết định nếu có thể.
- Thử thay số neuron 8, 32, 128 để thấy hiệu ứng capacity.
- Câu hỏi: mô hình nhiều tham số hơn có luôn tốt hơn không?

Không. Quá ít tham số có thể **underfitting**, nhưng quá nhiều tham số có thể **overfitting**, tốn tài nguyên và chưa chắc cải thiện dữ liệu mới.

Dữ liệu make_moons

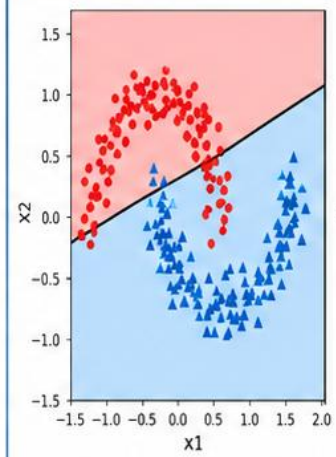
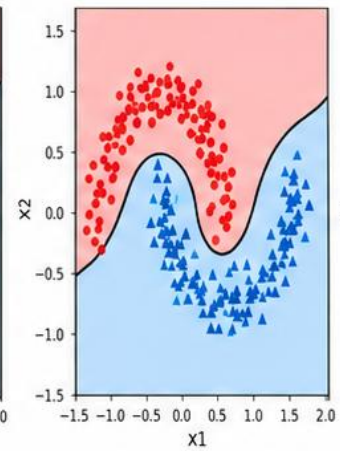
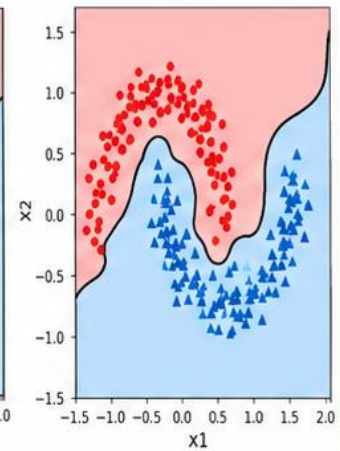


Ranh giới giữa hai lớp có dạng cong

Logistic Regression tuyến tính khó tách tốt

MLP học được ranh giới phi tuyến

Ảnh hưởng của số neuron

8 neuron	32 neuron	128 neuron
		
Dễ underfit	Cân bằng tốt	Có thể overfit

Điểm cần nhớ

1. Không chỉ nhìn **accuracy**
2. Cần quan sát thêm loss
3. Nếu có thể, xem ranh giới quyết định
4. Nhiều tham số hơn không phải lúc nào cũng tốt hơn

Errors

Vấn đề	Hiểu là:	Ví dụ
Nhầm batch dimension với feature dimension	batch là số mẫu xử lý cùng lúc, còn feature là số đặc trưng của mỗi mẫu	Shape [32, 784] nghĩa là có 32 ảnh, mỗi ảnh sau khi trải phẳng có 784 giá trị, không phải một ảnh có 32 đặc trưng
Quên flatten ảnh trước khi đưa vào MLP	MLP cần đầu vào dạng một dãy số, trong khi ảnh thường là ma trận nhiều chiều	Ảnh FashionMNIST có shape [1,28,28]; cần đổi thành [784] trước khi đưa vào MLP
Dùng activation sai ở lớp cuối với CrossEntropyLoss	CrossEntropyLoss đã tự xử lý bước biến đổi xác suất cần thiết, nên không cần thêm Softmax ở lớp cuối	Đúng: Linear(...,10) → CrossEntropyLoss; không nên: Linear(...,10) → Softmax → CrossEntropyLoss
Chỉ nhìn accuracy một lần rồi kết luận	Một con số chưa cho biết mô hình học có ổn định hay không	Accuracy 90% có vẻ tốt, nhưng nếu baseline đã 89% thì cải thiện rất ít; cần xem thêm loss train/validation theo từng epoch
Không cố định seed	Mỗi lần chạy có thể khởi tạo trọng số và chia dữ liệu khác nhau nên kết quả thay đổi	Lần 1 accuracy 88%, lần 2 là 91%; nếu đặt cùng seed, việc so sánh các mô hình sẽ công bằng và dễ lặp lại hơn